

1. Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP.

Use existing algorithms and measure the performance of sequential and parallel algorithms.

Bubble sort -

```
#include <iostream>           // Used for input and output
#include <omp.h>               // Used for OpenMP parallel programming
#include <ctime>               // Used for measuring execution time
using namespace std;         // Avoids writing std:: before cout, cin,
etc.

// Function to perform parallel bubble sort
void bubblesort(int a[], int n)
{
    // Outer loop runs n times to make sure array becomes fully
    sorted
    for (int i = 0; i < n; i++)
    {
        // first decides whether this pass starts from index 0 or index 1
        // If i is even, first = 0 → compare (0,1), (2,3), (4,5)
        // If i is odd, first = 1 → compare (1,2), (3,4), (5,6)
        int first = i % 2;

        // OpenMP parallel for creates multiple threads for the loop
        below
        // shared(a, first) means all threads use the same array and
        first value
        #pragma omp parallel for shared(a, first)

        // This loop compares alternate pairs
        // j increases by 2 because we compare non-overlapping pairs
        for (int j = first; j < n - 1; j += 2)
        {
            // If current element is greater than next element, swap them
            if (a[j] > a[j + 1])
            {
```

```

        // Store next element temporarily
        int temp = a[j + 1];

        // Move current larger element to next position
        a[j + 1] = a[j];

        // Put smaller element at current position
        a[j] = temp;
    }
}
}

```

```

// Print sorted array after sorting is completed
cout << "Sorted list: (Bubble Sort)\n";

```

```

// Loop to print all elements of sorted array
for (int i = 0; i < n; i++)
{
    cout << a[i] << " ";
}
}

```

```

// Main function starts here
int main()
{
    // Number of elements in the array
    int n = 100;

    // Variables used for storing start and end time
    clock_t t1, t2;

    // Print total number of elements
    cout << "Total number of elements: " << n << endl;

    // Create array of size n
    int a[n];

    // Inform user that array is stored in descending order

```

```

cout << "Storing elements in descending order....\n\n";

// Fill array in descending order
// Example: if n = 100, array becomes 100, 99, 98, ...
for (int i = 0; i < n; i++)
{
    a[i] = n - i;
}

// Print original array before sorting
cout << "Actual list:\n";

// Loop to print original array
for (int i = 0; i < n; i++)
{
    cout << a[i] << " ";
}

cout << "\n";

// Store time before sorting starts
t1 = clock();

// Call bubble sort function
bubblesort(a, n);

// Store time after sorting ends
t2 = clock();

// Print execution time
// Difference between t2 and t1 gives time taken by sorting
printf("\nExecution Time: %f", (double)(t2 - t1) /
CLOCKS_PER_SEC);

// End of program
return 0;
}

```

Output-

Total number of elements: 100

Storing elements in descending order....

Actual list:

100	99	98	97	96	95	94	93	92	91	90	89	88	87	86
85	84	83	82	81	80	79	78	77	76	75	74	73	72	71
70	69	68	67	66	65	64	63	62	61	60	59	58	57	56
55	54	53	52	51	50	49	48	47	46	45	44	43	42	41
40	39	38	37	36	35	34	33	32	31	30	29	28	27	26
25	24	23	22	21	20	19	18	17	16	15	14	13	12	11
10	9	8	7	6	5	4	3	2	1					

Sorted list: (Bubble Sort)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
19	20	21	22	23	24	25	26	27	28	29	30	31	32	33			
34	35	36	37	38	39	40	41	42	43	44	45	46	47	48			
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63			
64	65	66	67	68	69	70	71	72	73	74	75	76	77	78			
79	80	81	82	83	84	85	86	87	88	89	90	91	92	93			
94	95	96	97	98	99	100											

Execution Time: 0.000048

...Program finished with exit code 0

Press ENTER to exit console.

Merge sort -

```
#include<iostream>    // For input/output (cout)
#include<omp.h>        // For OpenMP parallel programming
using namespace std;

// Function to merge two sorted parts of the array
void merge(int arr[], int l, int m, int r)
{
    int i, j, k;        // i → left array, j → right array, k → main array
    index
```

```
int n1 = m - l + 1;      // Size of left subarray
int n2 = r - m;          // Size of right subarray
```

```
int L[n1], R[n2];        // Temporary arrays to store left and right
parts
```

```
// Copy elements from original array to left subarray
for (i = 0; i < n1; i++)
    L[i] = arr[l + i];
```

```
// Copy elements from original array to right subarray
for (j = 0; j < n2; j++)
    R[j] = arr[m + 1 + j];
```

```
i = 0;                    // Start index for left array
j = 0;                    // Start index for right array
k = l;                    // Start index for merged array
```

```
// Compare elements from both arrays and store smaller one
while (i < n1 && j < n2)
{
    if (L[i] <= R[j])      // If left element is smaller or equal
    {
        arr[k] = L[i];    // Put left element in main array
        i++;              // Move left index
    }
    else                  // If right element is smaller
    {
        arr[k] = R[j];    // Put right element in main array
        j++;              // Move right index
    }
    k++;                  // Move main array index
}
```

```
// Copy remaining elements of left array (if any)
while (i < n1)
{
    arr[k] = L[i];
```

```
    i++;  
    k++;  
}
```

```
// Copy remaining elements of right array (if any)
```

```
while (j < n2)
```

```
{  
    arr[k] = R[j];  
    j++;  
    k++;
```

```
}
```

```
}
```

```
// Recursive function for merge sort
```

```
void mergeSort(int arr[], int l, int r)
```

```
{
```

```
    if (l < r)                // Continue only if more than one element
```

```
{
```

```
    int m = l + (r - l) / 2; // Find middle index (safe way)
```

```
    // OpenMP: create parallel sections
```

```
    #pragma omp parallel sections
```

```
{
```

```
    // First section → sort left half
```

```
    #pragma omp section
```

```
{
```

```
        mergeSort(arr, l, m);
```

```
}
```

```
    // Second section → sort right half
```

```
    #pragma omp section
```

```
{
```

```
        mergeSort(arr, m + 1, r);
```

```
}
```

```
}
```

```
// Merge the two sorted halves
```

```
merge(arr, l, m, r);
```

```
}  
}
```

```
// Function to print array elements
```

```
void printArray(int A[], int size)
```

```
{
```

```
    int i;
```

```
    // Loop to print each element
```

```
    for (i = 0; i < size; i++)
```

```
        cout << A[i] << " ";
```

```
    cout << "\n";           // Move to next line
```

```
}
```

```
// Main function
```

```
int main()
```

```
{
```

```
    int n = 199;           // Number of elements
```

```
    clock_t t1, t2;       // Variables to measure time
```

```
    cout << "total number of elements: " << n << endl;
```

```
    int a[n];             // Declare array
```

```
    cout << "Storing elements in descending order....\n\n";
```

```
    // Fill array in descending order (worst case)
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        a[i] = n - i;      // Example: 199, 198, 197, ...
```

```
    }
```

```
    cout << "Actual list:\n";
```

```
    printArray(a, n);     // Print original array
```

```
    t1 = clock();         // Start time
```

```

mergeSort(a, 0, n - 1);    // Call merge sort

t2 = clock();              // End time

cout << "Sorted list:(Merge Sort)\n";
printArray(a, n);          // Print sorted array

// Print execution time
printf("\nExecution Time: %f", (double)(t2 - t1) /
CLOCKS_PER_SEC);

return 0;                  // End program
}

```

Output-

Actual list:

```

199 198 197 196 195 194 193 192 191 190 189 188 187 186 185
184 183 182 181 180 179 178 177 176 175 174 173 172 171 170
169 168 167 166 165 164 163 162 161 160 159 158 157 156 155
154 153 152 151 150 149 148 147 146 145 144 143 142 141 140
139 138 137 136 135 134 133 132 131 130 129 128 127 126 125
124 123 122 121 120 119 118 117 116 115 114 113 112 111 110 109
108 107 106 105 104 103 102 101 100 99 98 97 96 95 94 93 92 91
90 89 88 87 86 85 84 83 82 81 80 79 78 77 76 75 74 73 72 71 70
69 68 67 66 65 64 63 62 61 60 59 58 57 56 55 54 53 52 51 50 49
48 47 46 45 44 43 42 41 40 39 38 37 36 35 34 33 32 31 30 29 28
27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4
3 2 1

```

Sorted list:(Merge Sort)

```

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46
47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67
68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88
89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106
107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122
123 124 125 126 127 128 129 130 131 132 133 134 135 136 137

```


138 139 140 141 142 143 144 145 146 147 148 149 150 151 152
153 154 155 156 157 158 159 160 161 162 163 164 165 166 167
168 169 170 171 172 173 174 175 176 177 178 179 180 181 182
183 184 185 186 187 188 189 190 191 192 193 194 195 196 197
198 199

Execution Time: 0.000016

...Program finished with exit code 0
Press ENTER to exit console.